University of Science and Technology Chittagong (USTC)

Department of Computer Science and Engineering



Lab Report-2

Course Title: Computer Graphics and Image Processing Lab

Course Code: CSE 418

Submitted To

Umme Aiman Jannat

Lecturer

CSE, FSET, USTC

Submitted By

Name: **Bijon Chowdhury Prem**

Roll No: **23070165**

Registration No: **1071**

Batch: **41 (Section- B, CSE)**

7th Semester

Experiment 1: Implementation of Direct Line Algorithm

Source Code:

```
import matplotlib.pyplot as plt

# Input coordinates
x1 = int(input("Enter x1: "))
y1 = int(input("Enter y1: "))
x2 = int(input("Enter x2: "))
y2 = int(input("Enter y2: "))

# Direct Line Algorithm
points_x = []
points_y = []

dx = x2 - x1
dy = y2 - y1

steps = max(abs(dx), abs(dy))

if steps == 0:
    points_x.append(x1)
    points_y.append(y1)
else:
    x_increment = dx / steps
    y_increment = dy / steps

    x = x1
    y = y1

    for i in range(steps + 1):
        points_x.append(round(x))
        points_y.append(round(y))


      x += x_increment
      y += y_increment

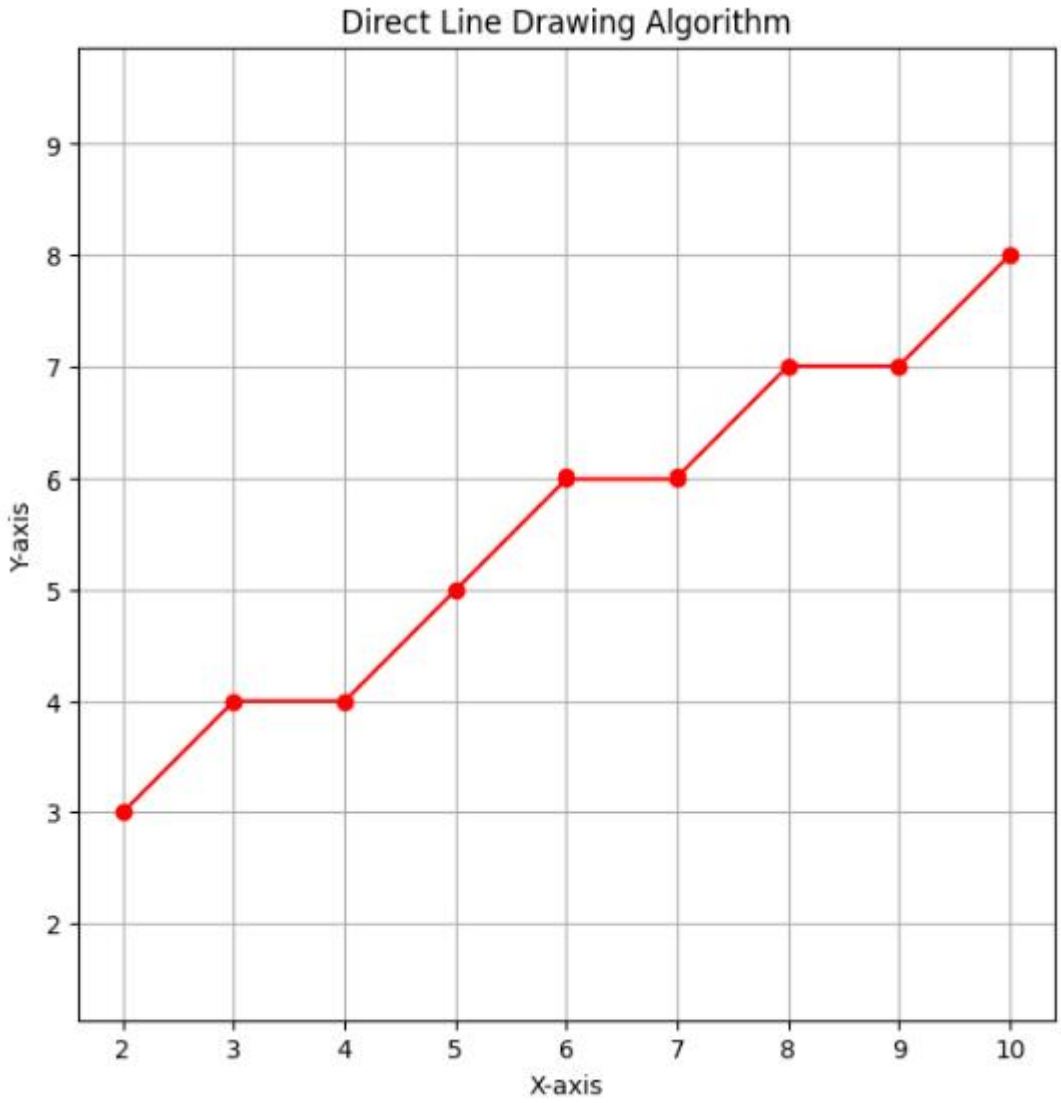
# Display output
plt.figure(figsize=(7, 7))
plt.plot(points_x, points_y, 'ro-')
plt.title("Direct Line Drawing Algorithm")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis('equal')
plt.show()

print("Generated Points:")
for i in range(len(points_x)):
    print(points_x[i], points_y[i])
```

Output:

```
Enter x1: 2
Enter y1: 3
Enter x2: 10
Enter y2: 8


 Generated Points:
 2 3
 3 4
 4 4
 5 5
 6 6
 7 6
 8 7
 9 7
 10 8
```



Experiment 2: Implementation of DDA Scan Conversion

Source Code:

```
import matplotlib.pyplot as plt

# Input coordinates
x1 = int(input("Enter x1: "))
y1 = int(input("Enter y1: "))
x2 = int(input("Enter x2: "))
y2 = int(input("Enter y2: "))

# Calculate differences
dx = x2 - x1
dy = y2 - y1

# Calculate steps
steps = max(abs(dx), abs(dy))

# Calculate increments
if steps == 0:
    x_points = [x1]
    y_points = [y1]
else:
    x_increment = dx / steps
    y_increment = dy / steps

    x = x1
    y = y1

    x_points = []
    y_points = []

    for i in range(steps + 1):
        x_points.append(round(x))
        y_points.append(round(y))

        x += x_increment
        y += y_increment

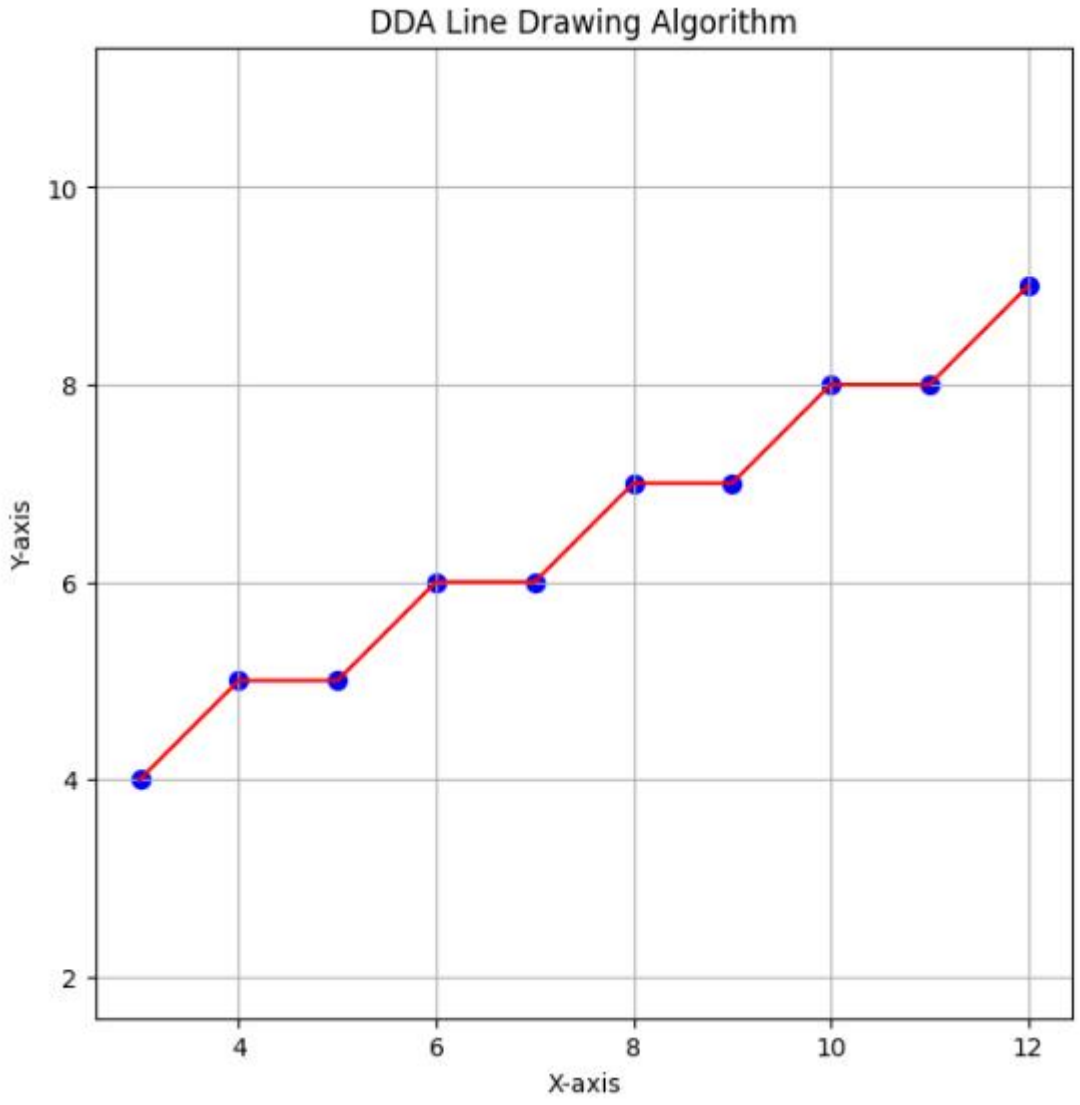
# Plot the line
plt.figure(figsize=(7, 7))
plt.scatter(x_points, y_points, color='blue', s=50)
plt.plot(x_points, y_points, color='red')
plt.title("DDA Line Drawing Algorithm")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis('equal')
plt.show()

print("DDA Generated Points:")
for x, y in zip(x_points, y_points):
    print(f"({x}, {y})")
```

Output:

```
Enter x1: 3
Enter y1: 4
Enter x2: 12
Enter y2: 9
```

```
DDA Generated Points:
(3, 4)
(4, 5)
(5, 5)
(6, 6)
(7, 6)
(8, 7)
(9, 7)
(10, 8)
(11, 8)
(12, 9)
```



Experiment 3: Implementation of Bresenham's Straight Line Algorithm.

Source Code:

```
import matplotlib.pyplot as plt

# Input coordinates
x1 = int(input("Enter x1: "))
y1 = int(input("Enter y1: "))
x2 = int(input("Enter x2: "))
y2 = int(input("Enter y2: "))

# Bresenham's Line Algorithm
points_x = []
points_y = []

dx = abs(x2 - x1)
dy = abs(y2 - y1)

sx = 1 if x1 < x2 else -1
sy = 1 if y1 < y2 else -1

err = dx - dy

while True:
    points_x.append(x1)
    points_y.append(y1)

    if x1 == x2 and y1 == y2:
        break

    e2 = 2 * err

    if e2 > -dy:
        err -= dy
        x1 += sx

    if e2 < dx:
        err += dx
        y1 += sy

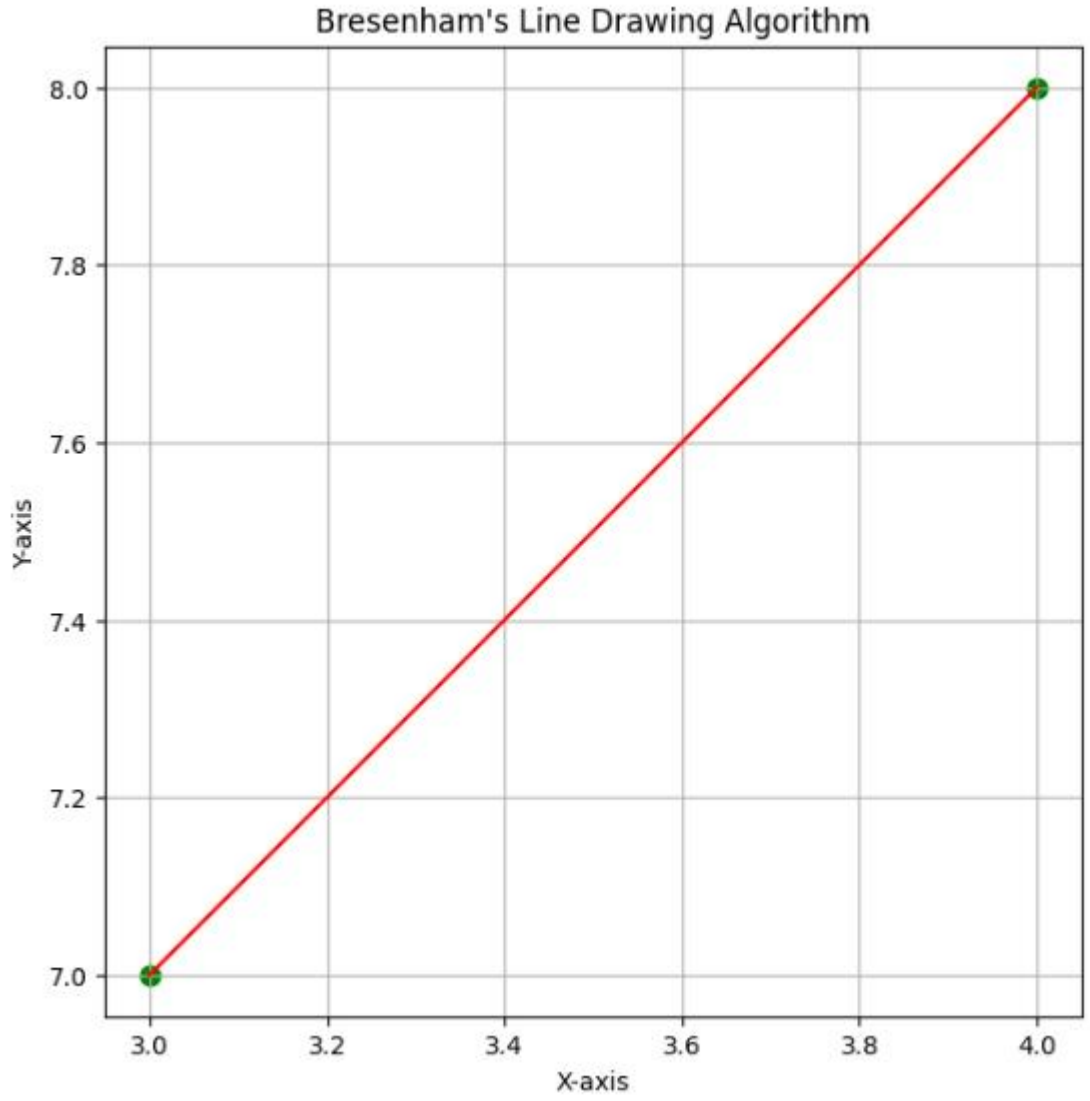
# Plot output
plt.figure(figsize=(7, 7))
plt.scatter(points_x, points_y, color='green', s=60)
plt.plot(points_x, points_y, color='red')
plt.title("Bresenham's Line Drawing Algorithm")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis('equal')
plt.show()

print("Generated Points:")
for x, y in zip(points_x, points_y):
    print(f"({x}, {y})")
```

Output:

```
Enter x1: 3
Enter y1: 7
Enter x2: 4
Enter y2: 8


Generated Points:
(3, 7)
(4, 8)
```



Experiment 4: Implementation of Bresenham's Circle Drawing Algorithm.

Source Code:

```
import matplotlib.pyplot as plt

# Input center and radius
xc = int(input("Enter center x: "))
yc = int(input("Enter center y: "))
r = int(input("Enter radius: "))

# Bresenham's Circle Algorithm
x = 0
y = r
d = 3 - 2 * r

points_x = []
points_y = []

def plot_circle_points(xc, yc, x, y):
    points_x.extend([
        xc + x, xc - x,
        xc + x, xc - x,
        xc + y, xc - y,
        xc + y, xc - y
    ])

    points_y.extend([
        yc + y, yc + y,
        yc - y, yc - y,
        yc + x, yc + x,
        yc - x, yc - x
    ])


while x <= y:
    plot_circle_points(xc, yc, x, y)

    if d < 0:
        d = d + 4 * x + 6
    else:
        d = d + 4 * (x - y) + 10
        y -= 1

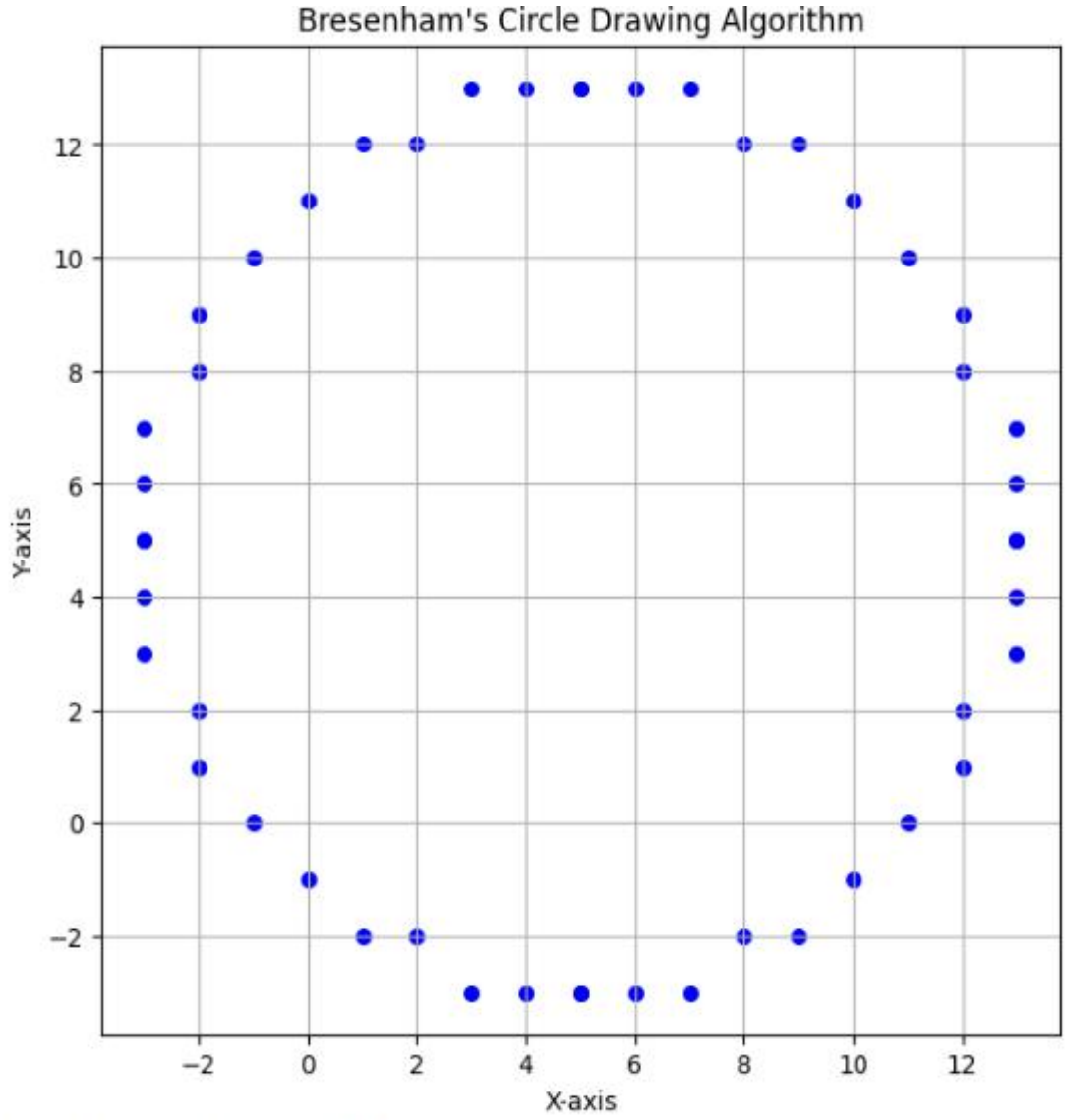
    x += 1

# Plot circle
plt.figure(figsize=(7, 7))
plt.scatter(points_x, points_y, color='blue', s=30)
plt.title("Bresenham's Circle Drawing Algorithm")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis('equal')
plt.show()

print("Circle generated successfully!")
```

Output:

```
Enter center x: 5
Enter center y: 5
Enter radius: 8
```




```
Circle generated successfully!
```